

Contrato de interfaz: Entradas / Inventario ↔ Pagos

Versión: 2.0 (Actualización a Flujo Asíncrono)

Equipo consumidor: Entradas / Inventario

Equipo proveedor: Pagos

Basado en: Revisión de viabilidad técnica y latencia de pasarela externa.

1. Propósito

El equipo de Entradas/Inventario (consumidor) necesita delegar el proceso de cobro al microservicio de Pagos (proveedor). Dado que el pago involucra la interacción del usuario y validaciones bancarias que exceden los tiempos de una petición HTTP normal, Entradas inicia la transacción vía REST, recibe un estado de "PENDIENTE", y luego espera pasivamente un evento en el broker (RabbitMQ) para consolidar la venta y emitir la entrada, o liberar el stock reservado.

2. Operación Síncrona: Iniciar Pago (iniciar_pago)

2.1 Descripción

Registra la intención de pago y devuelve un identificador de seguimiento con estado inicial PENDIENTE.

2.2 Quién la expone

Equipo Pagos.

2.3 Quién la consume

Equipo Entradas / Inventario, una vez que el stock temporal está bloqueado y el precio final calculado.

2.4 Endpoint propuesto

POST /api/v1/pagos/transacciones

2.5 Request (lo que se envía)

Campo	Tipo	Obligatorio	Descripción
id_reserva	string	Sí	Identificador de la reserva temporal generada por Entradas.

total	integer	Sí	Monto final a cobrar.
cantidad_entradas	integer	Sí	Cantidad de tickets en la orden.
token_sesion	string	Sí	Token del usuario comprador.

Ejemplo Body:

```
{
  "id_reserva": "res-98765",
  "total": 15000,
  "cantidad_entradas": 2,
  "token_sesion": "eyJhbGciOiJIUzI1NiIsInR5cCI6I.."}

```

2.6 Response (lo que se recibe)

Campo	Tipo	Obligatorio	Descripción
id_pago	string	Sí	Identificador único del pago en el sistema financiero.
estado_pago	string	Sí	Estado inicial de la transacción (Siempre PENDIENTE en este paso).

Ejemplo:

```
{
  "id_pago": "pay-112233",
  "estado_pago": "PENDIENTE"}

```

2.7 Códigos de error

Código	Significado
400	Datos de pago incompletos o monto inválido.
500	Error interno al intentar registrar la intención de pago.

2.8 Tiempo de respuesta esperado (SLA)

< 2000 ms (Aplica SOLO para la respuesta inicial de creación del pago en estado PENDIENTE, no para la resolución de la pasarela).

3. Integración Asíncrona (RabbitMQ) y Lógica de Reserva

Dado que la resolución bancaria toma tiempo, la confirmación definitiva se procesará vía broker:

1. Entradas escuchará los eventos emitidos por Pagos: "PagoAprobado" y "PagoRechazado".
2. El payload de estos eventos incluirá el "id_reserva" y el "id_pago" para que Entradas pueda enlazarlo con la compra en curso.
3. Lógica de retención: Como el pago puede durar hasta 10 minutos, Entradas configurará el tiempo de vida (TTL) de su stock reservado en al menos 12 a 15 minutos para evitar que la reserva caduque mientras el usuario está introduciendo sus datos bancarios.
4. Si Entradas recibe "PagoAprobado", emite la entrada. Si recibe "PagoRechazado", libera inmediatamente el stock reservado.

4. Operación de Fallback: Consultar Estado (Opcional)

Endpoint expuesto por Pagos: GET /api/v1/pagos/{id_pago}

Uso: Si Entradas no recibe un evento por el broker después del tiempo máximo de espera (ej. 15 minutos), puede consultar proactivamente este endpoint para conocer el estado final antes de tomar la decisión de liberar o consolidar el stock.

5. Dueños del contrato

Rol	Equipo	Contacto
Dueño del contrato	Pagos	[Representante de Pagos]
Consumidor principal	Entradas / Inventario	Sebastián Fuentes (Scrum Master)